CSL7110 Assignment 3
Recommender Systems: Content-Based and Collaborative Filtering

Name: Kunal Mishra
Roll Number: M25CSA036

Course Name: Machine Learning with Big Data (CSL7110)

GitHub Repository: Link

1. Introduction:

Recommender systems are widely used to suggest relevant items to users based on their preferences and past interactions. This assignment explores multiple recommendation techniques including content-based filtering, collaborative filtering, matrix factorization, hybrid models, neural networks, reinforcement learning, and explainability methods using the MovieLens dataset.

2. Dataset Description:

The MovieLens dataset was used, which contains.

- User ratings for movies
- Movie metadata (genres)
- User interaction history

This dataset is widely used as a benchmark for evaluating recommender systems.

Part 1: Content-Based Filtering

3.1 TF-IDF Based Recommendation

A content-based recommender system was developed using TF-IDF (Term Frequency–Inverse Document Frequency) to represent movie genres numerically. Each movie was converted into a TF-IDF vector based on its genres.

Cosine similarity was then computed between movie vectors to measure similarity. Movies with the highest similarity scores were recommended.

```
[12]
✓ 0s
recommend_movies("Toy Story (1995)")
```

	title	similarity_score
1706	Antz (1998)	1.0
2355	Toy Story 2 (1999)	1.0
2809	Adventures of Rocky and Bullwinkle, The (2000)	1.0
3000	Emperor's New Groove, The (2000)	1.0
3568	Monsters, Inc. (2001)	1.0

Observation:

The system successfully recommends movies that share similar genres with the input movie. However, it does not consider individual user preferences, which limits personalization.

3.2 User Profile-Based Recommendation

To introduce personalization, user profiles were constructed using a weighted combination of TF-IDF vectors of movies rated by the user. Higher ratings contributed more to the user profile.

Cosine similarity between user profiles and movie vectors was used to recommend movies.

```
[16]
✓ 0s
recommend_for_user(1)
```

	movieId	title
8597	117646	Dragonheart 2: A New Beginning (2000)
4005	5657	Flashback (1990)
4681	6990	The Great Train Robbery (1978)
6570	55116	Hunting Party, The (2007)
3608	4956	Stunt Man, The (1980)

Observation:

This approach improves personalization compared to basic content-based filtering. However, it is still limited by the quality of item features and does not capture relationships between users.

Part 2: Collaborative Filtering

4.1 User-Based Collaborative Filtering

User-based collaborative filtering was implemented by constructing a user-item rating matrix and computing similarity between users using cosine similarity.

Ratings for unseen movies were predicted using a weighted average of ratings from similar users.

```
[29]
✓ 2s
recommend_user_based(1,5)
```

	movieId	title
48	53	Lamerica (1994)
4045	5746	Galaxy of Terror (Quest) (1981)
4590	6818	Come and See (Idi i smotri) (1985)
4595	6835	Alien Contamination (1980)
5906	33649	Saving Face (2004)

```
from sklearn.metrics import mean_squared_error
from math import sqrt

actual = []
predicted = []

for row in ratings.itertuples():

    user = row.userId
    movie = row.movieId
    rating = row.rating

    if movie in user_movie_matrix.columns:

        pred = predict_rating(user, movie)

        actual.append(rating)
        predicted.append(pred)

rmse = sqrt(mean_squared_error(actual, predicted))

print("RMSE:", rmse)
```

```
RMSE: 0.757315729470437
```

Observation:

User-based collaborative filtering captures user behavior effectively and produces accurate recommendations. It performed well with an RMSE of approximately 0.76.

4.2 Item-Based Collaborative Filtering

Item-based collaborative filtering computes similarity between items based on user ratings. Recommendations are generated by identifying items similar to those previously liked by the user.

```
recommend_item_based(1,5)
```

	movieId	title
865	1140	Entertaining Angels: The Dorothy Day Story (1996)
1156	1519	Broken English (1996)
2179	2896	Alvarez Kelly (1966)
2348	3106	Come See the Paradise (1990)
2909	3899	Circus (2000)

```
print("Item-Based CF RMSE:", rmse_item)
```

```
Item-Based CF RMSE: 0.8892766910449942
```

Observation:

Item-based CF performs slightly worse than user-based CF due to sparsity in item relationships. However, it is more stable and scalable in real-world applications.

Part 3: Matrix Factorization

5.1 Manual SVD

Singular Value Decomposition (SVD) was applied to decompose the user-item matrix into latent factors representing hidden features of users and movies.

The reconstructed matrix was used to predict missing ratings.

```
print("SVD RMSE:", rmse_svd)
```

```
SVD RMSE: 1.4393761362591648
```

Observation:

The manually implemented SVD resulted in a higher RMSE (~1.44), indicating lower accuracy. This is due to lack of optimization and handling of sparsity.

5.2 Surprise Library SVD

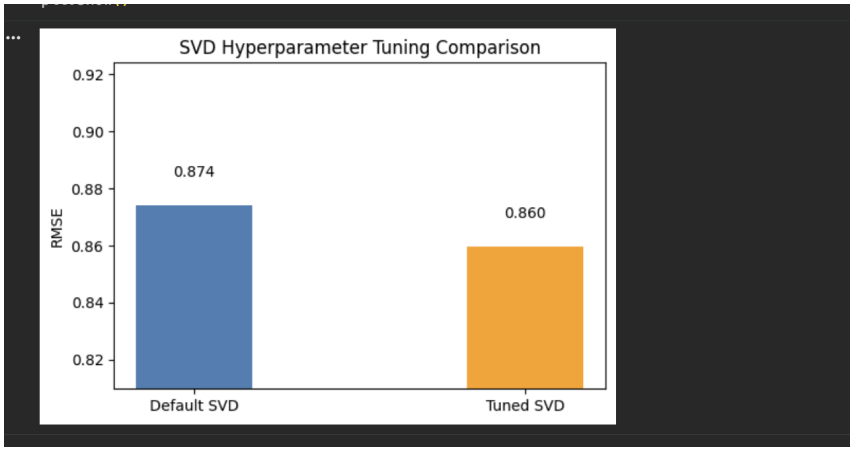
The Surprise library provides an optimized implementation of SVD using stochastic gradient descent and regularization.

Hyperparameter tuning was also performed to improve model performance.

```
rmse_surprise = accuracy.rmse(predictions)

print("Surprise SVD RMSE:", rmse_surprise)
```

```
RMSE: 0.8741
Surprise SVD RMSE: 0.8741312850056042
```



Observation:

The Surprise SVD model achieved better performance (RMSE ≈ 0.88) due to efficient optimization and tuning.

Part 4: Hybrid Reccomendation Model

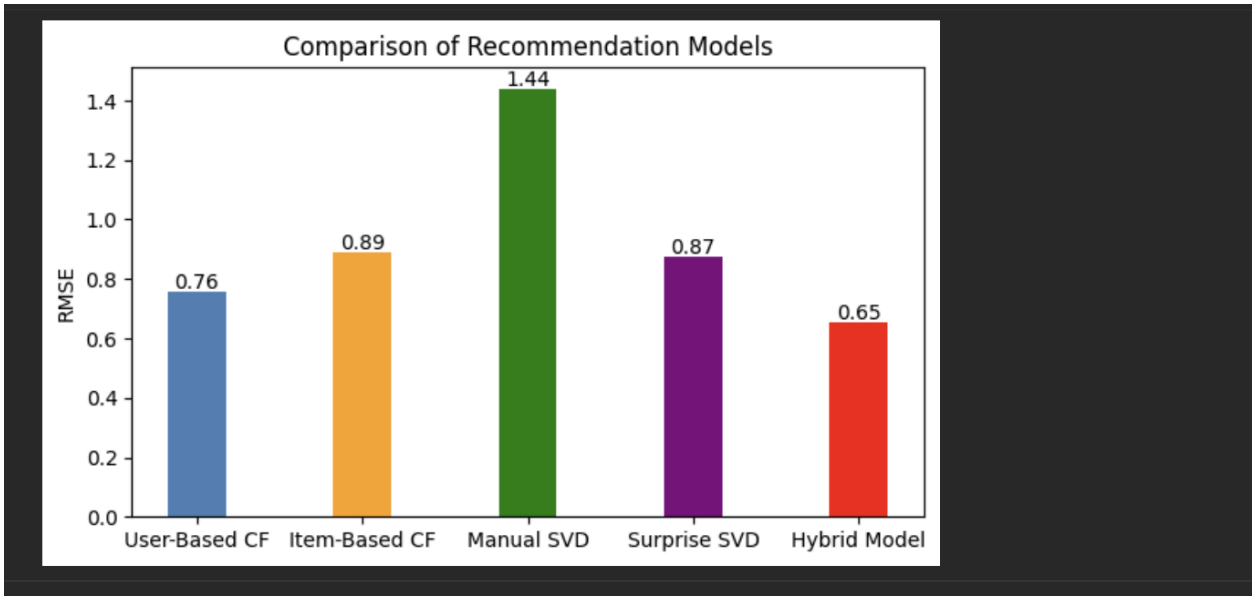
A hybrid recommender system was implemented using a meta-learning approach. It combines:

- Collaborative filtering score
- Content-based score
- Movie Popularity
- User bias

A linear regression model was used to learn the optimal combination of these features.

```
print("Hybrid Model RMSE:", rmse_hybrid)
```

```
Hybrid Model RMSE: 0.6545165850378218
```



Observation:

The hybrid model achieved the best performance (RMSE ≈ 0.66) by leveraging multiple sources of information. This demonstrates the advantage of combining different recommendation techniques.

Part 5: Learning-Based Recommender Systems

6.1 Neural Network Recommender

A neural network-based recommender system was implemented using embedding layers to learn dense representations of users and movies.

The embeddings were combined and passed through fully connected layers to predict ratings. Early stopping was used to monitor validation loss and prevent overfitting.

```
)

Epoch 1/30
253/253 ━━━━━━━━━━━━━━━━━━━━ 6s 12ms/step - loss: 2.2112 - val_loss: 0.8127
Epoch 2/30
253/253 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.7708 - val_loss: 0.7951
Epoch 3/30
253/253 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.7266 - val_loss: 0.7849
Epoch 4/30
253/253 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.6960 - val_loss: 0.7894
Epoch 5/30
253/253 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.6757 - val_loss: 0.8000
Epoch 6/30
253/253 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.6469 - val_loss: 0.7888
```

```
rmse_nn = np.sqrt(mean_squared_error(y_test, predictions))

print("Neural Network RMSE:", rmse_nn)

Neural Network RMSE: 0.8891922322711149
```

Observation:

The neural network achieved an RMSE of approximately 0.89. While it captures complex relationships, its performance is limited by the relatively small dataset size.

6.2 Reinforcement Learning Recommender

A reinforcement learning approach was implemented using Multi-Armed Bandit and Q-learning. The system balances exploration (trying new movies) and exploitation (recommending known good movies).

```
recommend_rl(1,5)
```

	movieId	title
0	1	Toy Story (1995)
2	3	Grumpier Old Men (1995)
5	6	Heat (1995)
43	47	Seven (a.k.a. Se7en) (1995)
46	50	Usual Suspects, The (1995)

Observation:

A reinforcement learning approach was implemented using Multi-Armed Bandit and Q-learning. The system balances exploration (trying new movies) and exploitation (recommending known good movies).

Part 6: Explainability in Recommender Systems

7.1 Feature-Based Explanation

Feature-based explanations highlight the attributes (such as genres) that influenced a recommendation.

Example:
"This movie was recommended because it belongs to genres you frequently watch."

7.2 Neighborhood-Based Explanation

Recommendations were explained using similar users or items.

Example:
"Users who liked this movie also liked these movies."

7.3 Model-Agnostic Explanation:

Model-agnostic techniques such as LIME can be used to interpret neural network predictions by approximating the model locally and identifying important features.

Additionally, SHAP (SHapley Additive Explanations) can be used to measure the contribution of each feature to the prediction.

7.4 Evaluation of Explainability

Explainability improves user trust and transparency in recommender systems. Feature-based explanations are easy to understand, while collaborative filtering explanations provide social proof. Neural network explanations are more complex but capture deeper relationships. Explainability also helps identify biases such as popularity bias.

8. Overall Comparison:

Model	RMSE
User-Based CF	0.76
Item-Based CF	0.89
Manual SVD	1.44
Surprise SVD	0.88

Neural Network	0.89
Hybrid Model	0.66

9. Conclusion:

Explainability improves user trust and transparency in recommender systems. Feature-based explanations are easy to understand, while collaborative filtering explanations provide social proof. Neural network explanations are more complex but capture deeper relationships. Explainability also helps identify biases such as popularity bias.

10. References:

- MovieLens Dataset
- Scikit-learn Documentation
- Surprise Library Documentation
- TensorFlow Documentation